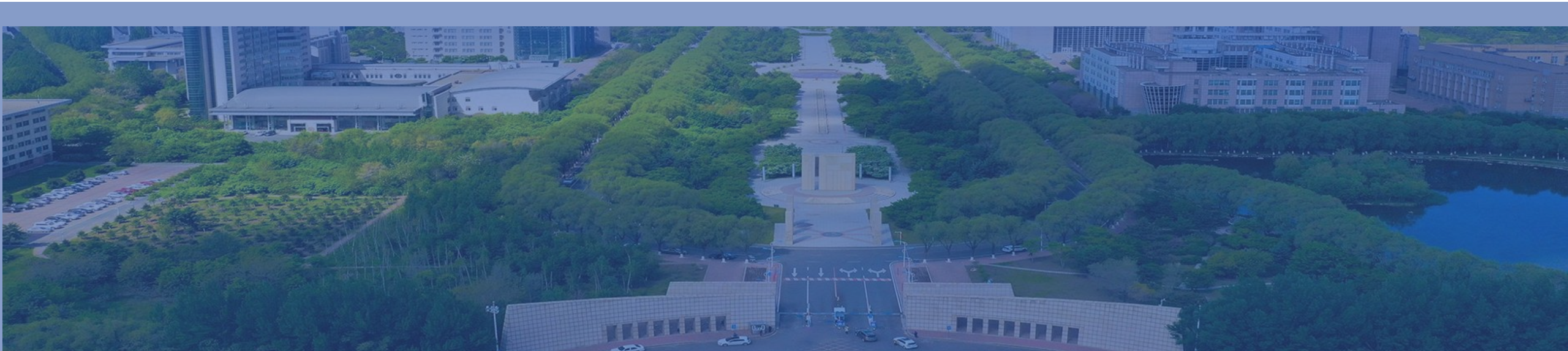


吉林大学



算法分析

任课教师：王元宏



课程信息

- 理论课
 - 时间：周三**5-7**节
 - 周数：**14**周（第**14**周仅第**5**节）
 - 地点：敬信楼**D301**
- 实验课
 - 时间：3、6、8、10、12、14周
 - 地点：正新楼**3**楼机房



课程内容

- 基本的算法分析方法

如何判断算法的优劣？

- 基本的算法设计策略

如何设计算法？

- 分治法
- 贪心方法
- 动态规划
- 回溯法
- 分支-限界法

- NP-难度和NP-完全问题



课程考核方式

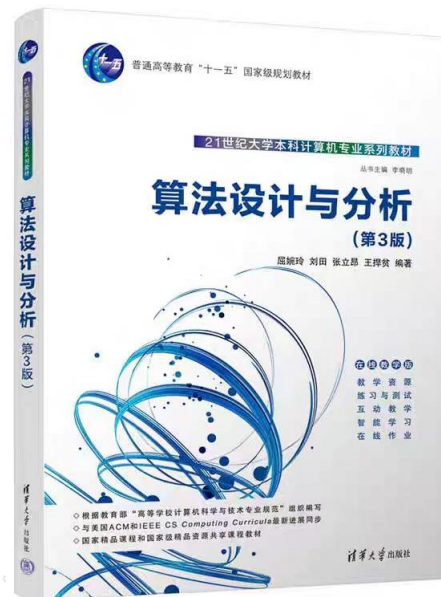
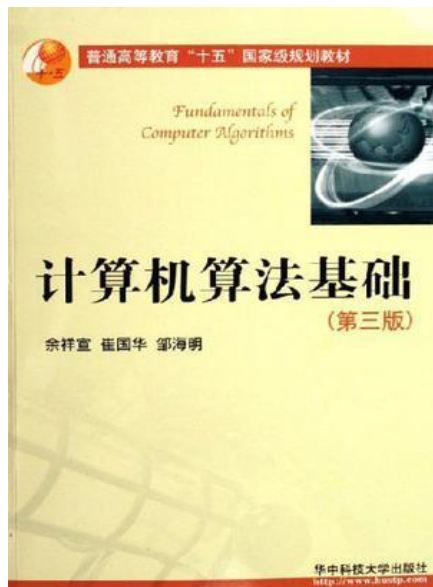
- 52学时：40理论学时 + 12实验学时
- 平时考核（占总成绩30%）
 - 学习任务（实验2x6）
 - 课堂测试/练习（10）
 - 课外作业：4次（2x4）
- 期末考核（占总成绩70%）
 - 闭卷笔试





参考书目

- 余祥宣等. 计算机算法基础(第三版) 华中科技大学出版社, 2005
- 屈婉玲等. 算法设计与分析(第三版) 清华大学出版社, 2011
- 托马斯·科尔曼 等. 算法导论(原书第3版) 机械工业出版社, 2013



什么是算法？

- 生活中的算法

衣



食



美团

饿了么



大众点评
dianping.com

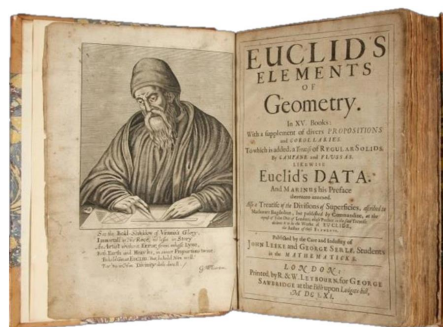
住



行



算法的由来

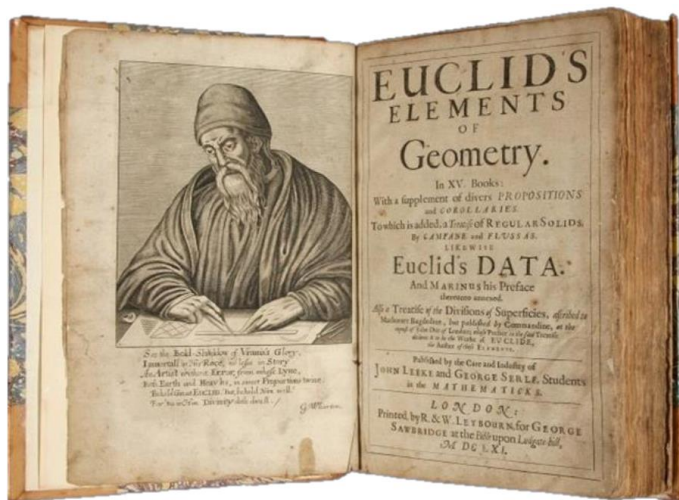


约公元前300年
辗转相除法
《几何原本》

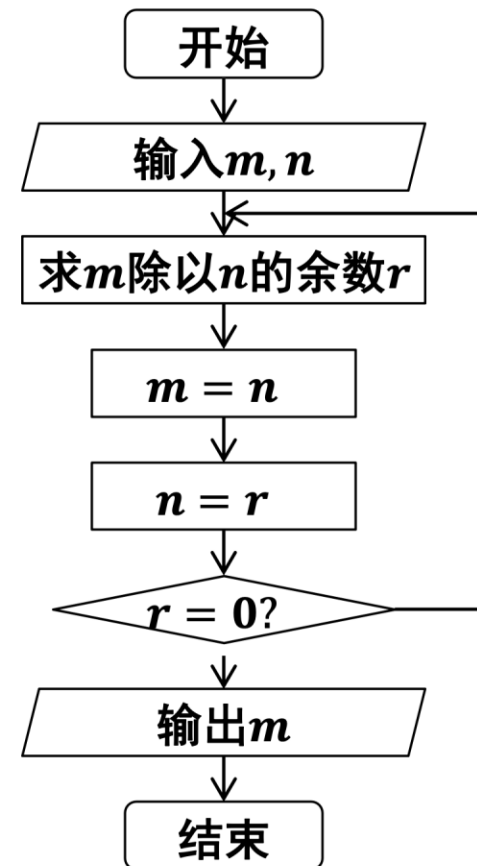


算法的由来

- 辗转相除法
 - 计算两个整数的最大公约数
 - 约公元前**300**年由欧几里德提出



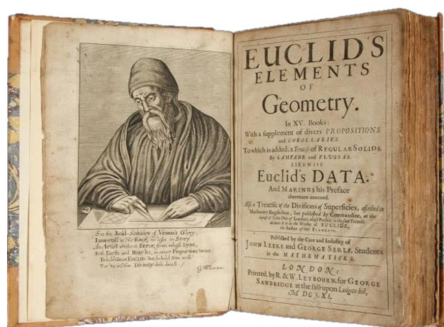
欧几里得《几何原本》



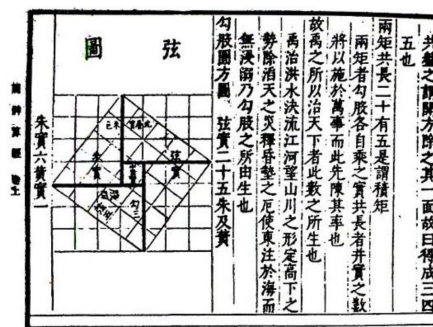
辗转相除法流程图



算法的由来



约公元前300年
辗转相除法
《几何原本》

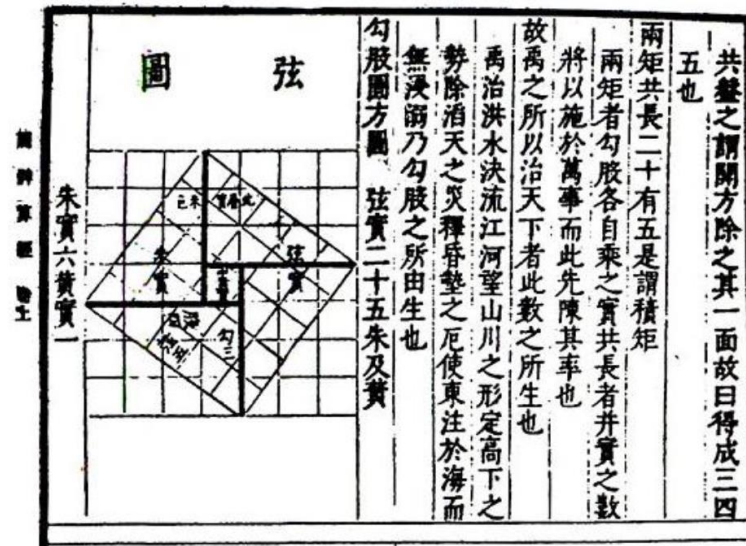


约公元前1世纪
《周髀算经》

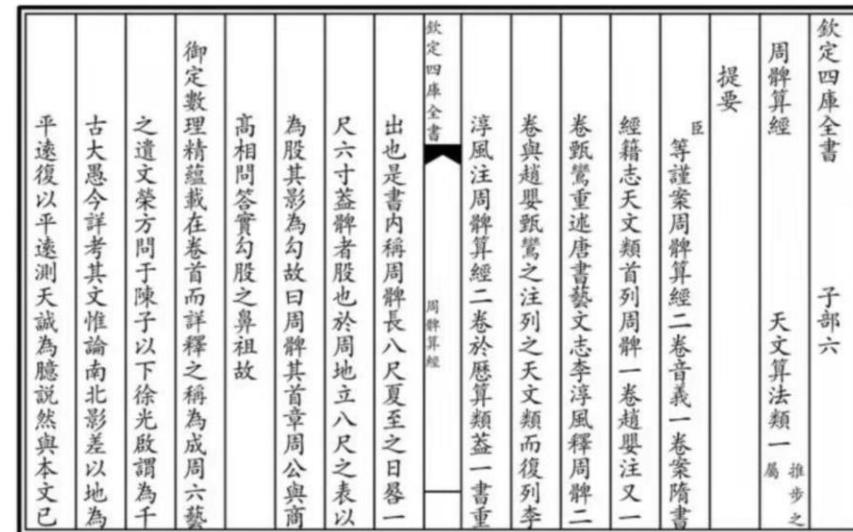


算法的由来

- 中文名称“算法”出自约成书于公元前1世纪的《周髀算经》
 - 勾股定理：“以日下为勾，日高为股，勾股各自乘，并而开方除之”
 - 采用简便可行方法确定天文历法揭示日月星辰运行规律

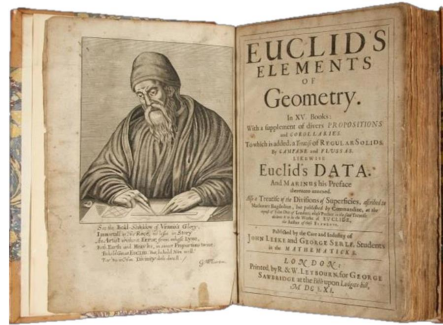


勾股定理

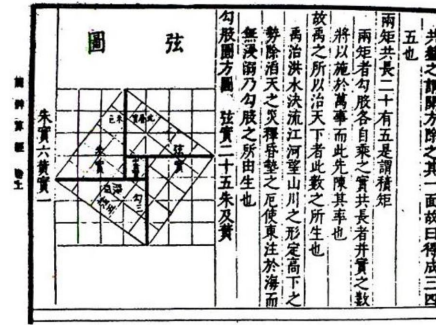


确定天文历法

算法的由来



约公元前300年
《几何原本》
辗转相除法



约公元前1世纪
《周髀算经》
勾股定理



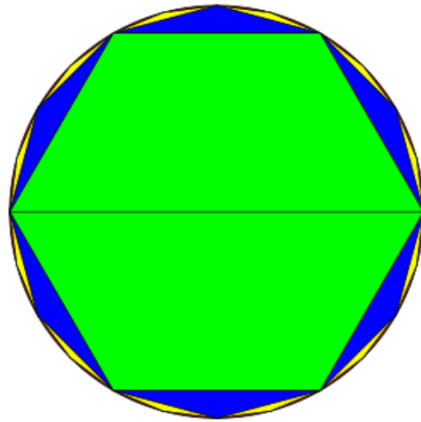
魏晋时期
《九章算术注》
割圆术



算法的由来

● 割圆术

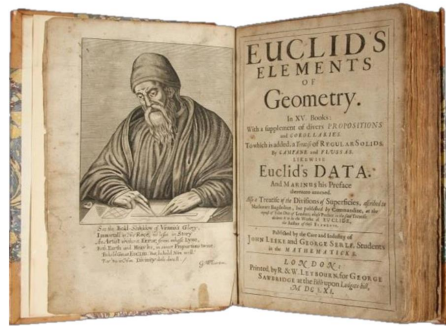
- 魏晋时期的数学家刘徽在《九章算术注》中首创
- 内接正多边形去无限逼近圆，并以此求取圆周率的方法



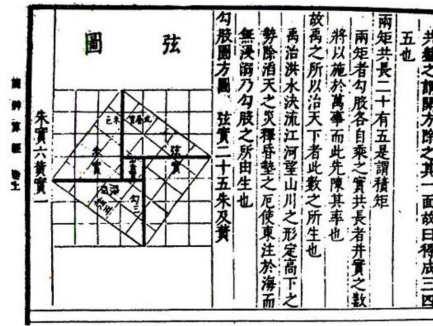
割圆术示意图



算法的由来



约公元前300年
《几何原本》
辗转相除法



约公元前1世纪
《周髀算经》
勾股定理



魏晋时期
《九章算术注》
割圆术



7世纪
《算经十书》



算法的由来

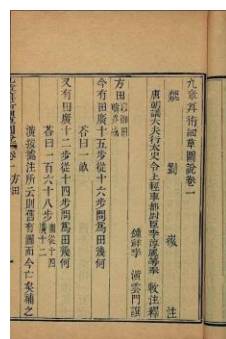
● 算经十书

- 唐高宗显庆元年（公元656年），规定将十部汉、唐一千多年间的十部著名数学著作作为国家最高学府的算学教科书，后世通称为《算经十书》

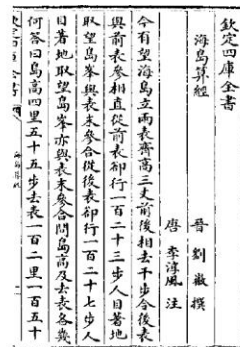
周髀算经



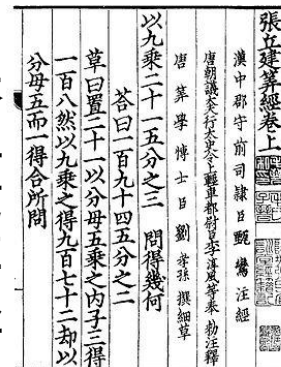
九章算术



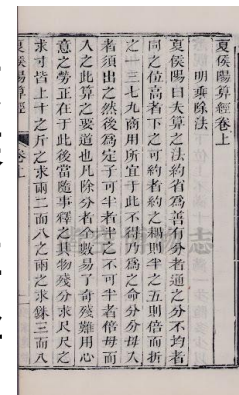
海岛算经



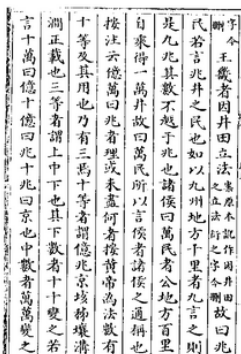
张丘建算经



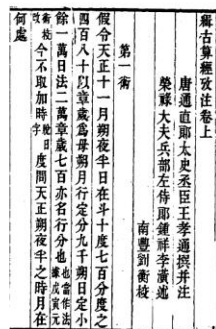
夏侯阳算经



五经算术



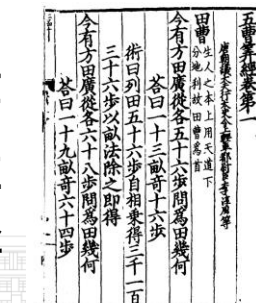
缉古算经



缀术



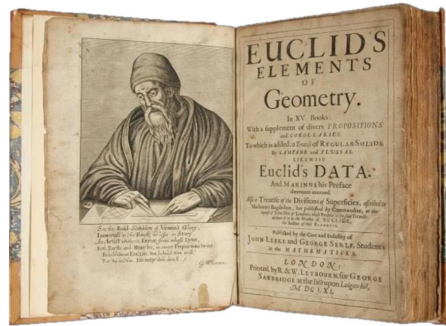
五曹算经



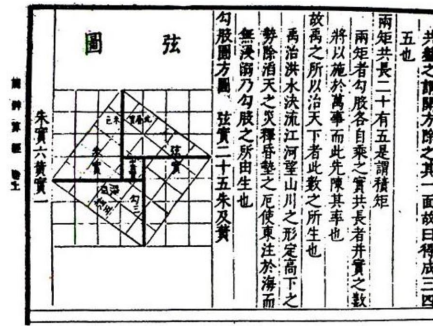
孙子算经



算法的由来



约公元前300年
《几何原本》
辗转相除法



约公元前1世纪
《周髀算经》
勾股定理



魏晋时期
《九章算术注》
割圆术



7世纪
《算经十书》



9世纪
Algorithm
阿尔·花拉子密



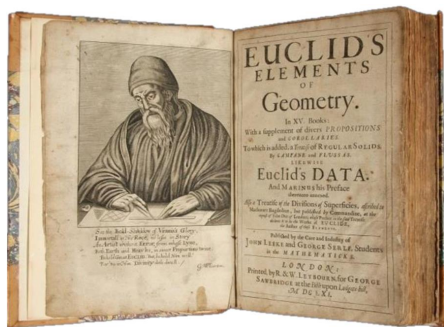
算法的由来

- 名称来源

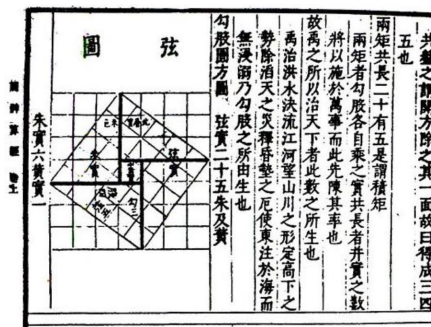
- 波斯著名的数学家、天文学家、地理学家阿尔·花拉子密在公元825年写成《印度数字算术》一书，对于印度-阿拉伯数字系统在中东及欧洲的传播起到了重要作用
- 该书被翻译成拉丁语 “Algoritmi de numero Indorum”，花拉子密的拉丁文音译即为 “算法”（Algorithm）一词的由来



算法的由来



约公元前300年
《几何原本》
辗转相除法



约公元前1世纪
《周髀算经》
勾股定理



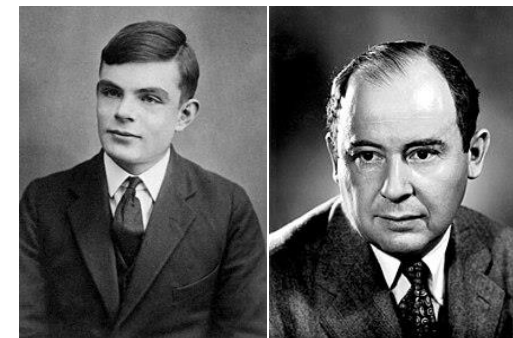
魏晋时期
《九章算术注》
割圆术



7世纪
《算经十书》



9世纪
Algorithm
阿尔·花拉子密

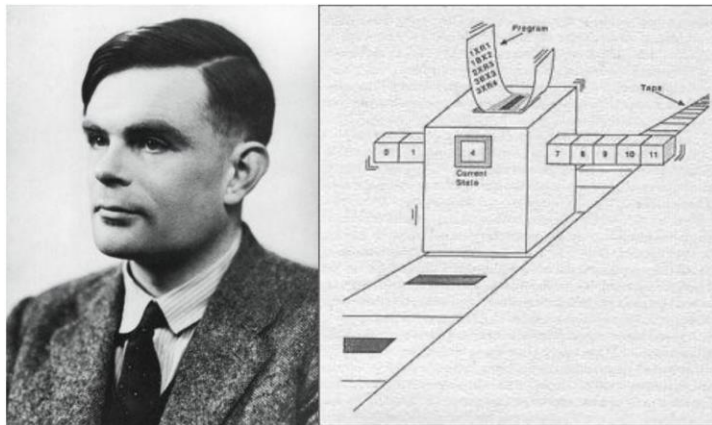


20世纪30年代~40年代
Alan Turing和John
von Neumann

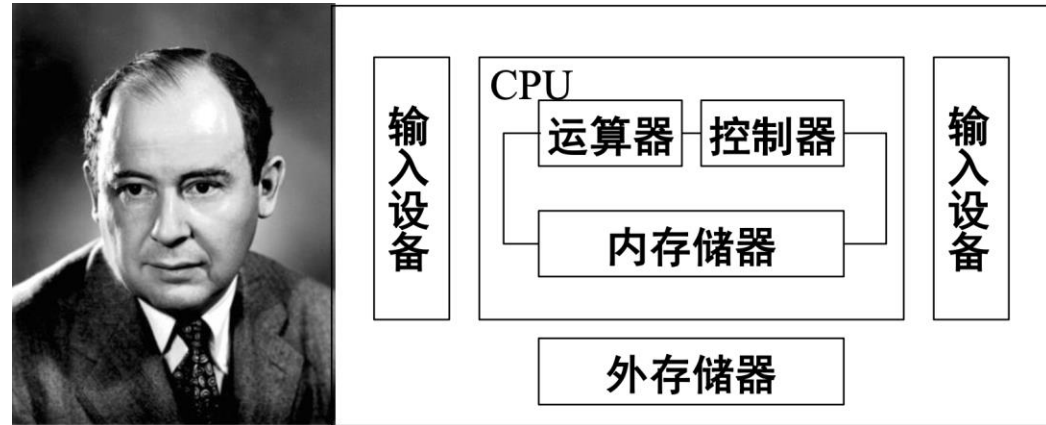
算法的由来

● 算法与计算机的结合

- 1936年，艾伦·图灵提出**图灵机**，通过建立通用计算机模型，刻画计算机的计算行为
- 1946年，冯·诺依曼提出**存储程序原理**



Alan Turing
理论计算机科学与人工智能之父



John von Neumann
现代计算机之父

- **1966**年由计算机协会(**ACM**)设立，奖励对计算机事业做出突出贡献的个人



ACM图灵奖
计算机界的“诺贝尔奖”



算法与计算复杂性领域图灵奖得主



Donald E. Knuth
1974, USA
算法分析之父



Michael O. Rabin
1976, Israeli
非确定自动机
素数判定随机算法



Dana S. Scott
1976, USA
非确定自动机



Robert W. Floyd
1978, USA
最短路径Floyd算法



Stephen A. Cook
1982, USA
NP完全性



Richard M. Karp
1985, USA
NP完全性与
网络流算法



John Hopcroft
1986, USA
最差情况分析
数据结构与算法



Robert Tarjan
1986, USA
数据结构与图算法



Juris Hartmanis
1993, Latvia
计算复杂性理论



Richard E. Stearns
1993, USA
计算复杂性理论



Manuel Blum
1995, Venezuela
计算复杂性理论



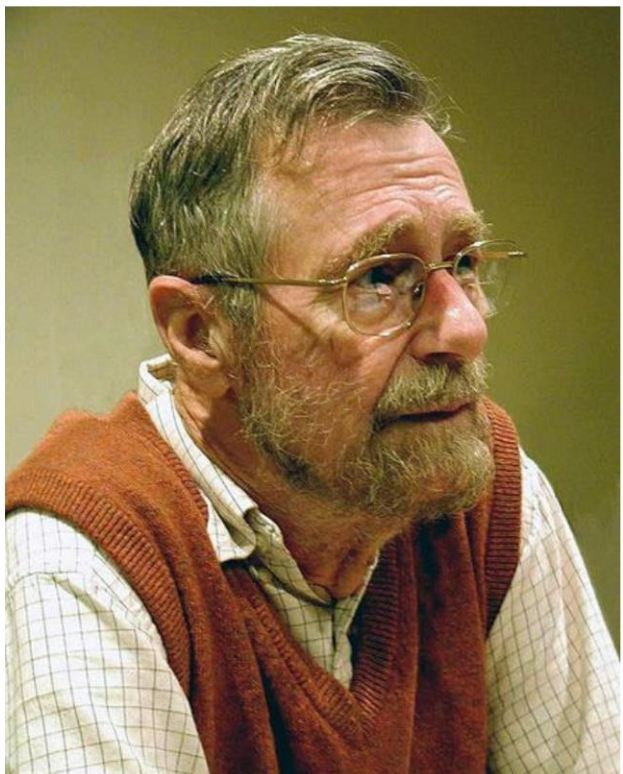
Andrew Yao
2000, China
伪随机数生成
与通信复杂性



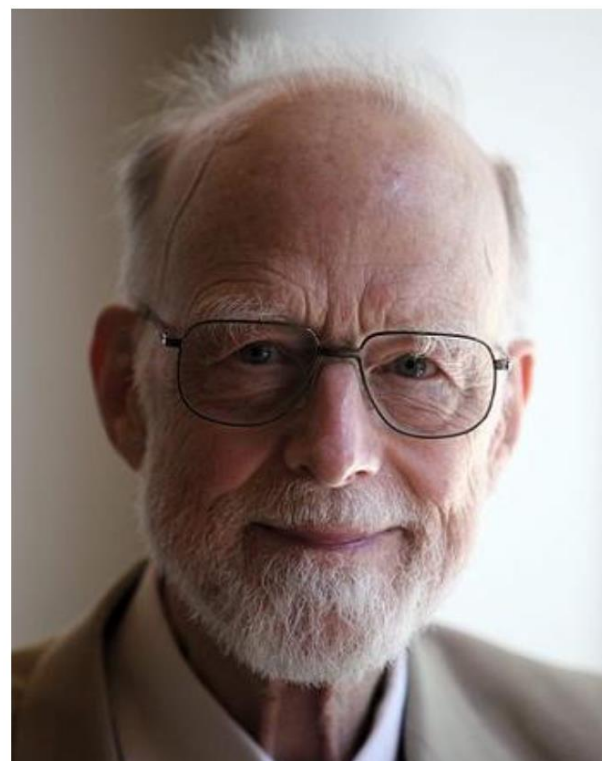
Leslie G. Valiant
2010, Hungarian
#P完全性与
计算学习理论

算法与计算复杂性领域图灵奖得主共计13人，占比约18.6%

其他相关图灵奖得主



Edsger W. Dijkstra
1972, Netherlands
ALGOL之父
提出单源最短路径Dijkstra算法



Tony Hoare
1980, UK
霍尔逻辑
提出快速排序算法



为什么学习算法？

- 受过良好训练的计算机科学家知道怎样处理算法：如何构造算法、操作算法、理解算法、分析算法。
- 这些知识远不止为了写出良好的计算机程序而准备的。算法是一种一般性的职能工具，它必定有助于我们对其他学科的理解，不管是化学、语言学、音乐，还是其他学科。

——Donald Knuth



为什么学习算法？

内核：算法在专业中的地位

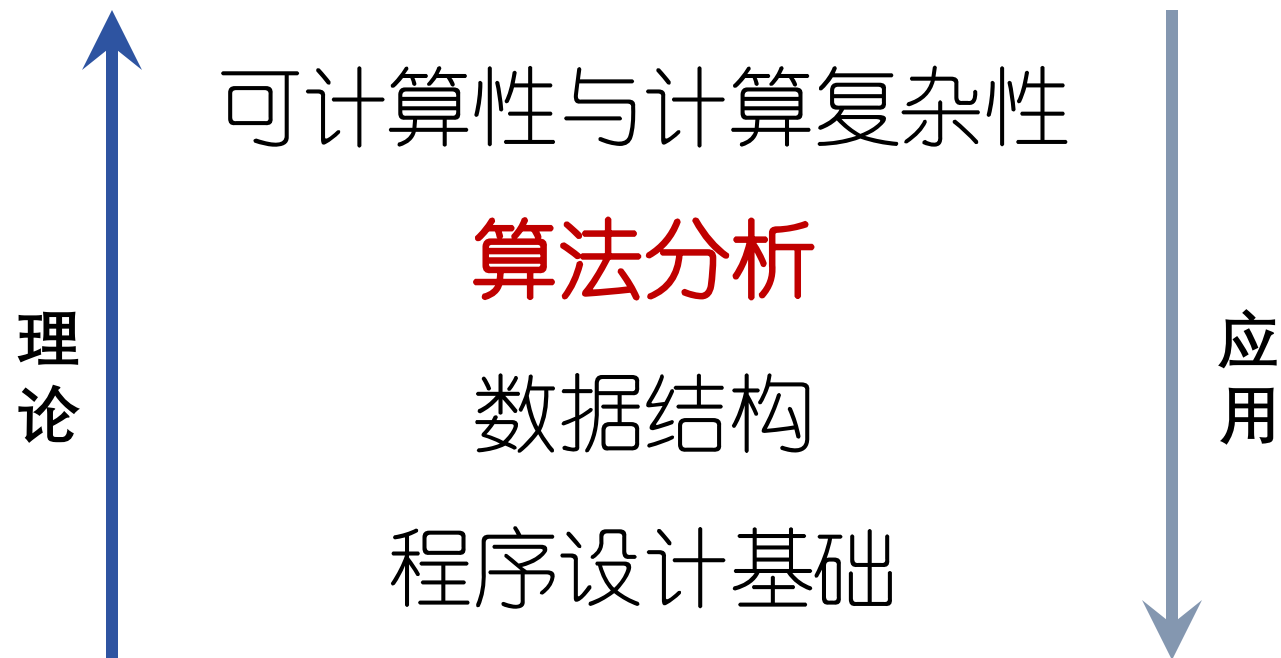


- 算法不只是计算机科学的一个分支，它是计算机科学的核心。而且，可以毫不夸张地说，它与绝大多数科学、商业和技术都密切相关。

—— “Algorithmics: the spirit of computing”

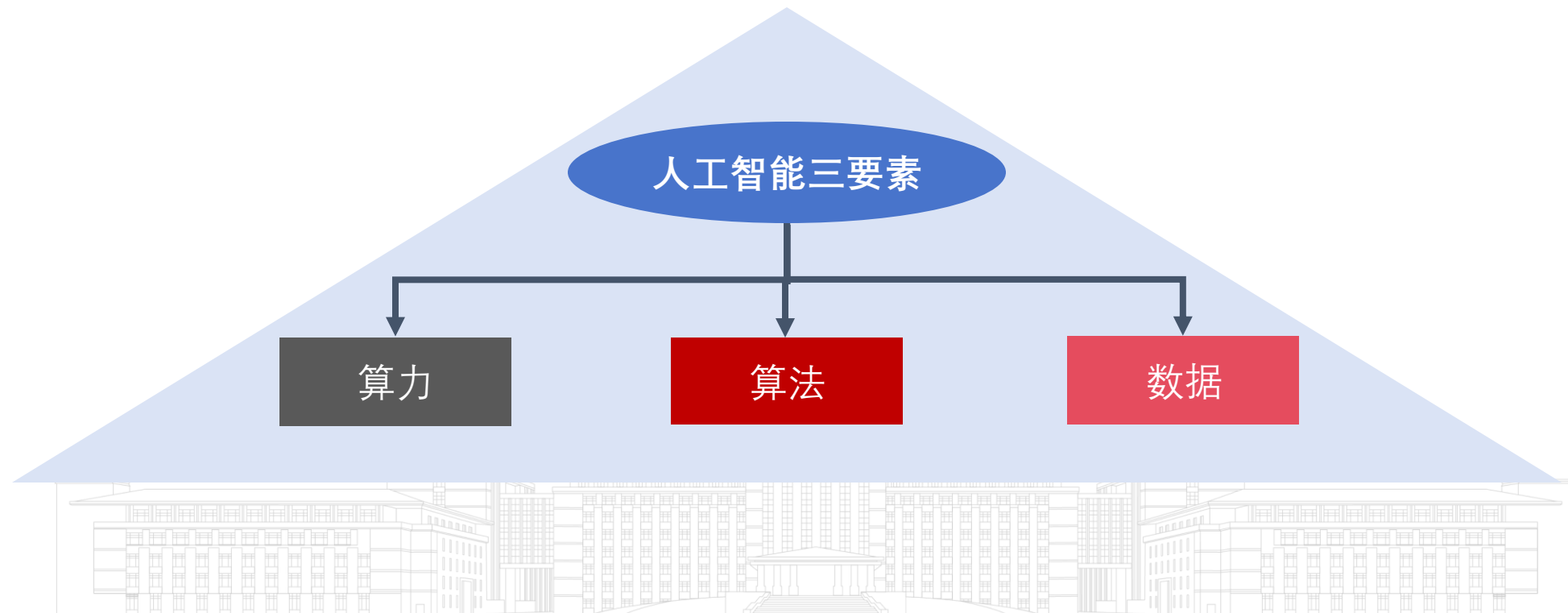


涉及算法的课程



人工智能中的算法

- 人工智能：用计算机程序来呈现人类智能的技术——学习、解决问题和决策
- 如何实现？**算法！**

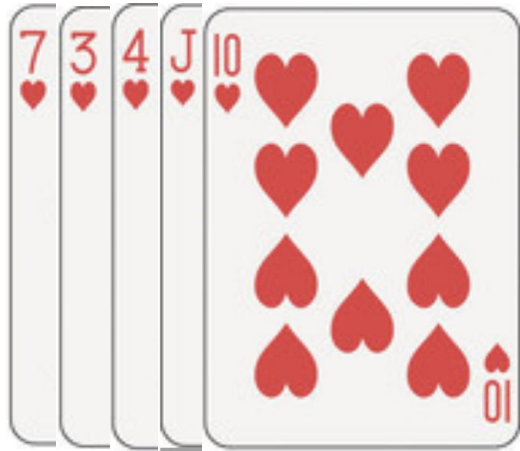


打扑克中的算法



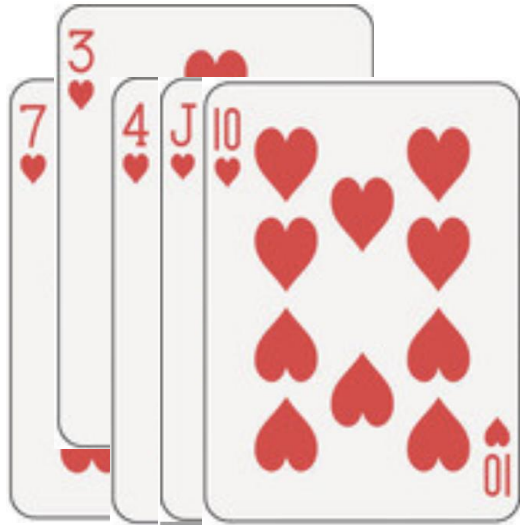
打扑克中的算法

- 如何整理手牌？



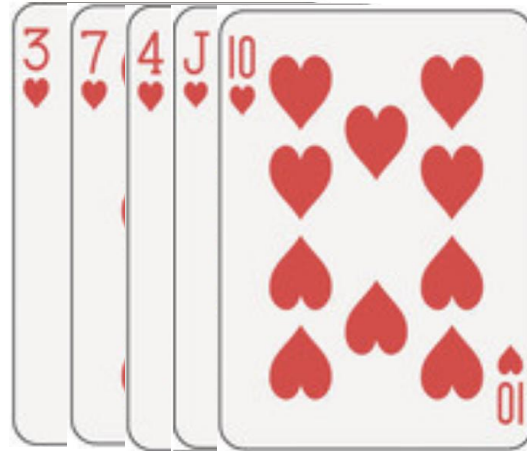
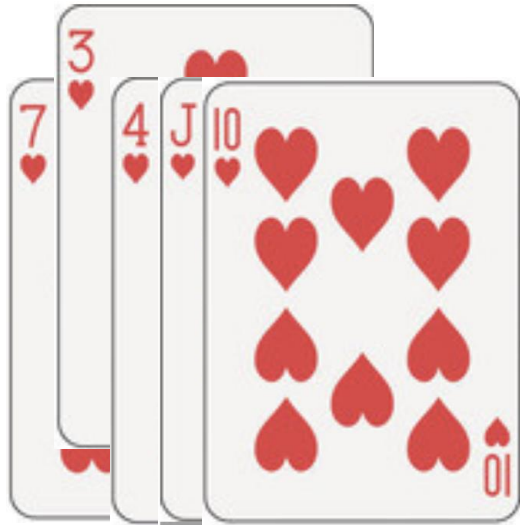
打扑克中的算法

- 如何整理手牌？



打扑克中的算法

- 如何整理手牌？



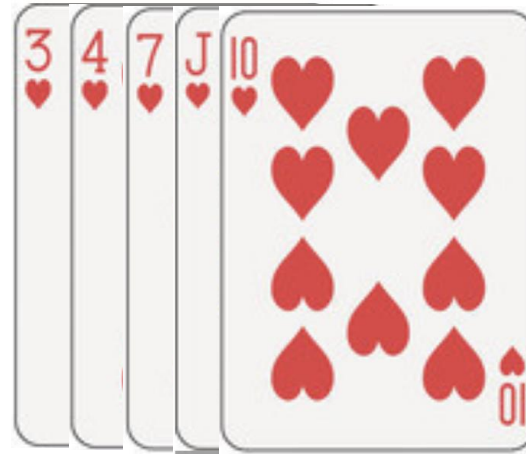
打扑克中的算法

- 如何整理手牌？



打扑克中的算法

- 如何整理手牌？



打扑克中的算法

- 如何整理手牌？



打扑克中的算法

- 如何整理手牌？



什么是算法？

- 计算问题定义

- 给定数据输入，计算满足某种性质（可机械验证）输出的问题



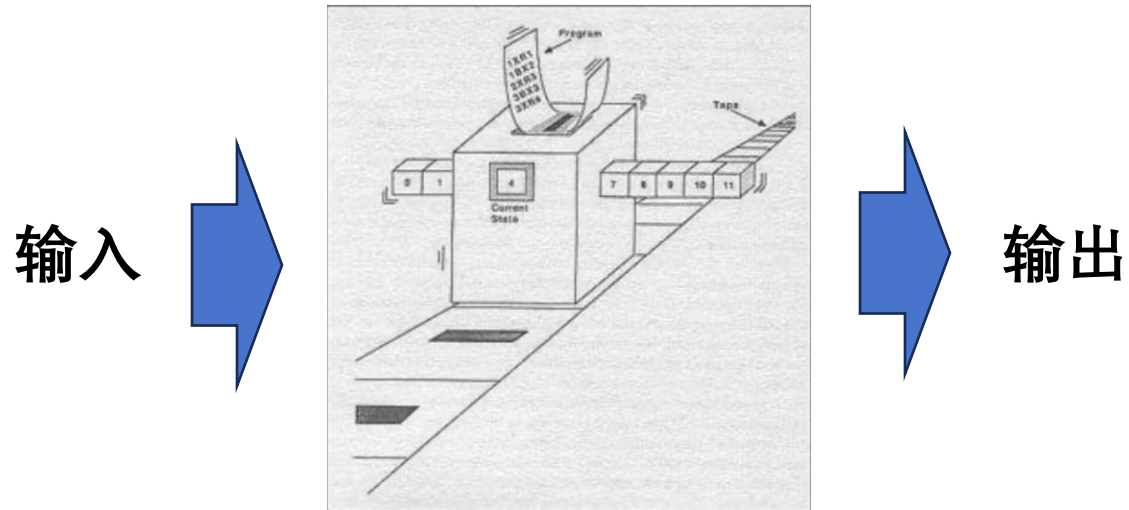
什么是算法？

- 计算问题定义
 - 给定数据输入，计算满足某种性质（可机械验证）输出的问题
- 手牌整理问题抽象（排序问题）
 - 输入：包含 n 个数字的数组 $[a_1, a_2, \dots, a_n]$
 - 升序排列的数组 $[a_1', a_2', \dots, a_n']$ ，满足 $a_1' \leq a_2' \leq \dots \leq a_n'$



什么是算法？

- 给定计算问题

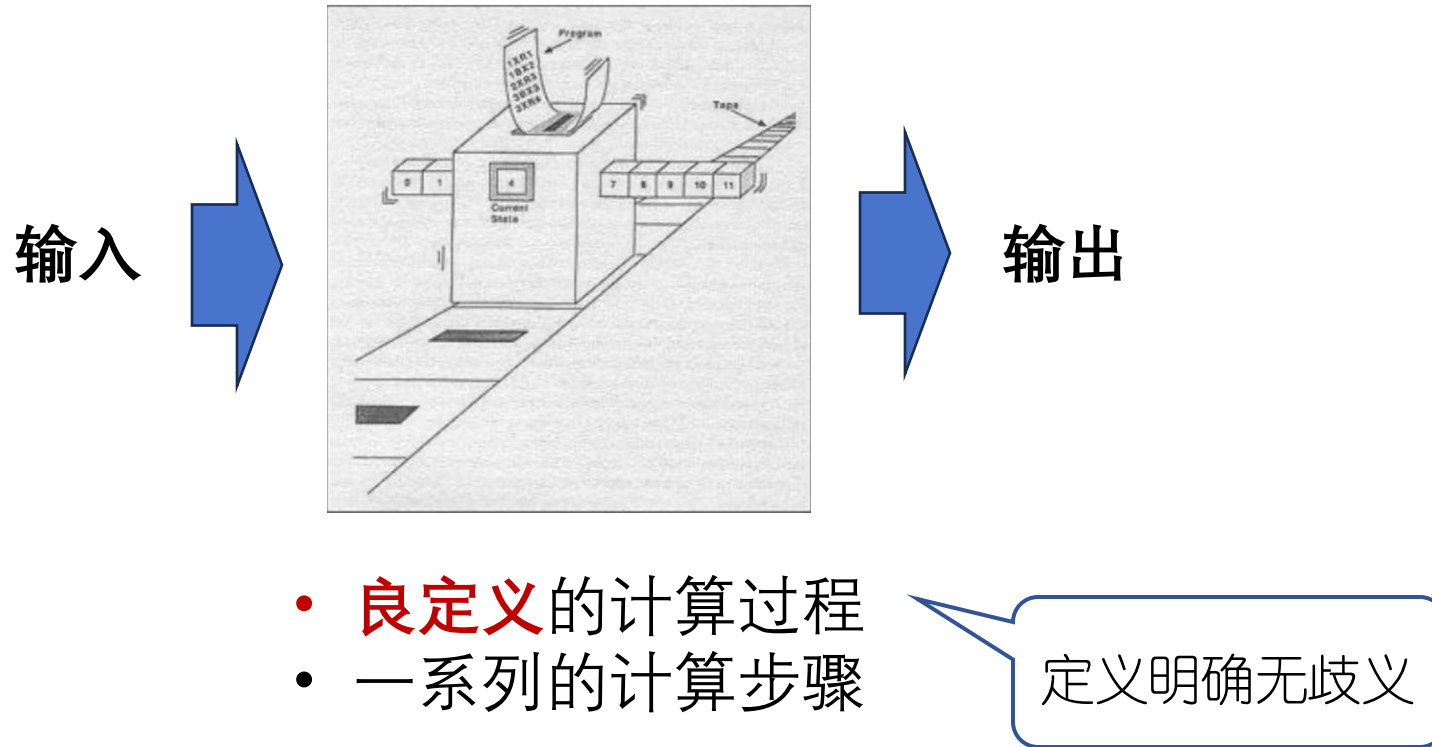


- 良定义的计算过程
- 一系列的计算步骤



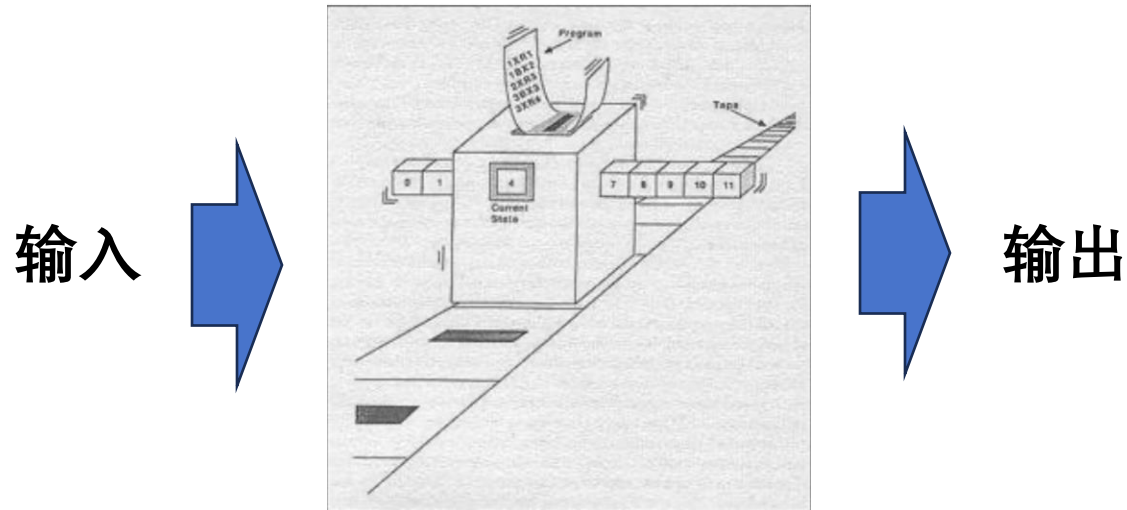
什么是算法？

- 给定计算问题



什么是算法？

- 给定计算问题



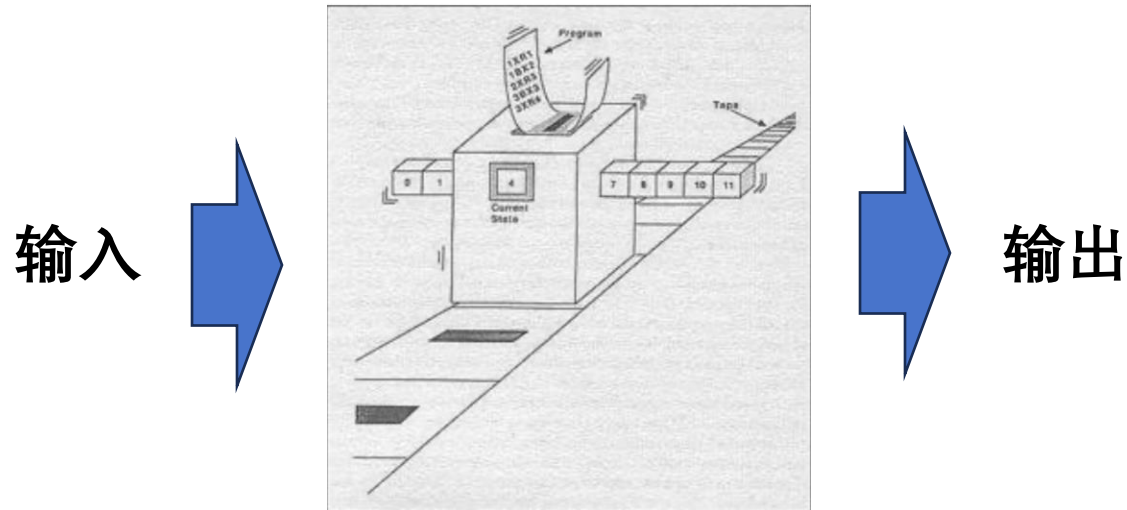
- 良定义的计算过程
- 一系列的步骤

计算步骤有限



什么是算法？

- 给定计算问题



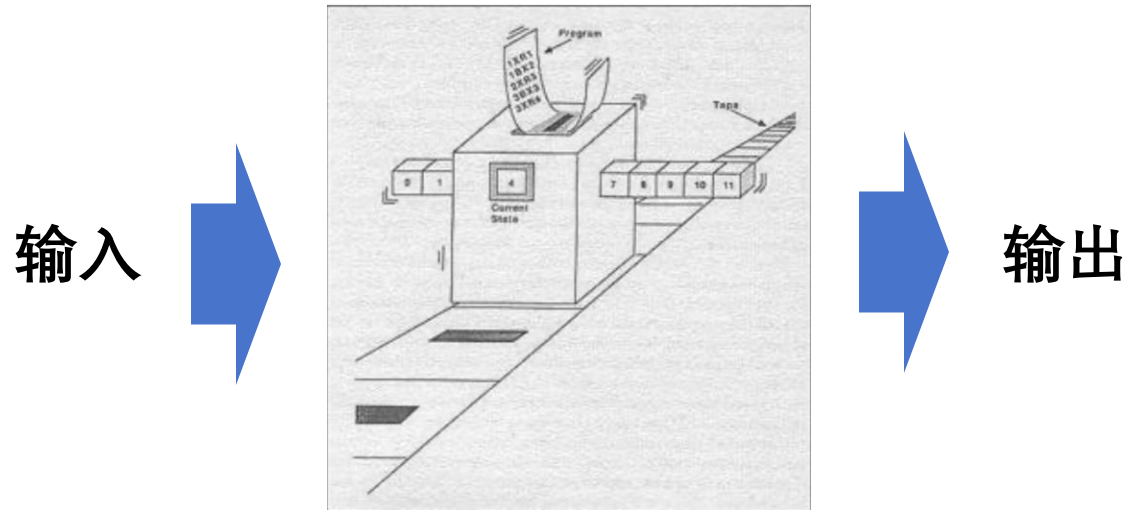
- 良定义的计算过程
- 一系列的**计算步骤**

计算机可实现的
指令



什么是算法？

- 给定计算问题



- 良定义的计算过程
- 一系列的**计算步骤**

算法正确 $\Leftrightarrow$ 任意输入实例, 算法**终止** + 结果**正确**

插入排序

- 输入: [7, 3, 4, 11, 10]

7	3	4	11	10
---	---	---	----	----



插入排序



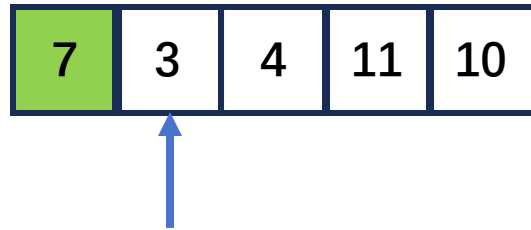
- 输入: [7, 3, 4, 11, 10]

7	3	4	11	10
---	---	---	----	----



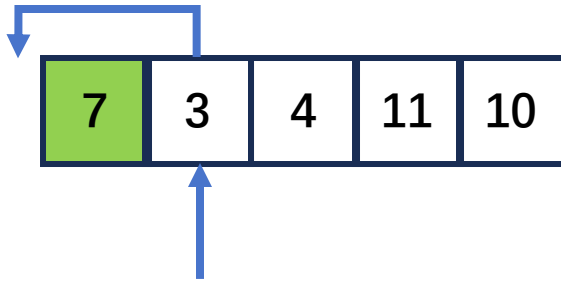
插入排序

- 输入：[7, 3, 4, 11, 10]



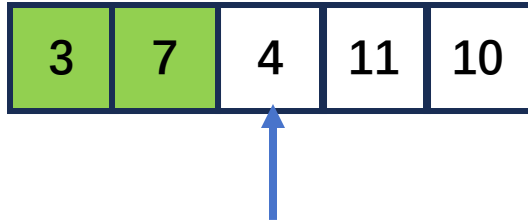
插入排序

- 输入：[7, 3, 4, 11, 10]



插入排序

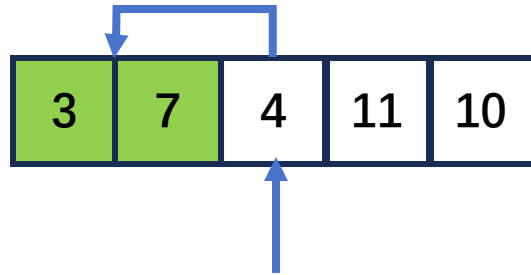
- 输入: [7, 3, 4, 11, 10]



插入排序

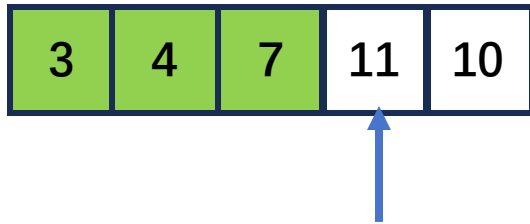


- 输入: [7, 3, 4, 11, 10]



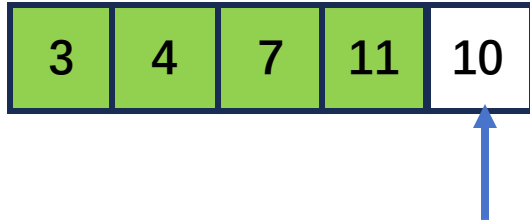
插入排序

- 输入: [7, 3, 4, 11, 10]



插入排序

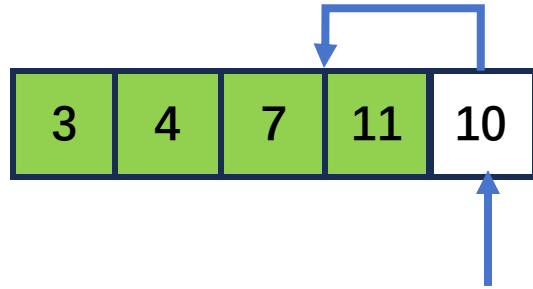
- 输入: [7, 3, 4, 11, 10]



插入排序



- 输入: [7, 3, 4, 11, 10]



插入排序

- 输入: [7, 3, 4, 11, 10]

3	4	7	10	11
---	---	---	----	----



插入排序

- 输入: [7, 3, 4, 11, 10]
- 输出: [3, 4, 7, 10, 11]

3	4	7	10	11
---	---	---	----	----



插入排序

- 输入：[7, 3, 4, 11, 10]
- 输出：[3, 4, 7, 10, 11]

3	4	7	10	11
---	---	---	----	----

思考：排序问题是否存在其他算法？



选择排序

- 输入：包含 n 个数字的数组 $[a_1, a_2, \dots, a_n]$
- 输出： $[a_1', a_2', \dots, a_n']$ ，满足 $a_1' \leq a_2' \leq \dots \leq a_n'$
- 选择排序：
 - 第一次遍历找到最小元素
 - 第二次在剩余数组中遍历找到次小元素
 - ...
 - 第 n 次在剩余数组中遍历找到第 n 小元素



选择排序



- 输入：<24, 17, 40, 28, 13, 14, 22, 32, 40, 21, 48, 4, 47, 8, 37, 18>

24	17	40	28	13	14	22	32	40	21	48	4	47	8	37	18
----	----	----	----	----	----	----	----	----	----	----	---	----	---	----	----



选择排序



- 输入：<24, 17, 40, 28, 13, 14, 22, 32, 40, 21, 48, 4, 47, 8, 37, 18>

24	17	40	28	13	14	22	32	40	21	48	4	47	8	37	18
----	----	----	----	----	----	----	----	----	----	----	---	----	---	----	----

↑



选择排序



- 输入：<24, 17, 40, 28, 13, 14, 22, 32, 40, 21, 48, 4, 47, 8, 37, 18>

24	17	40	28	13	14	22	32	40	21	48	4	47	8	37	18
----	----	----	----	----	----	----	----	----	----	----	---	----	---	----	----

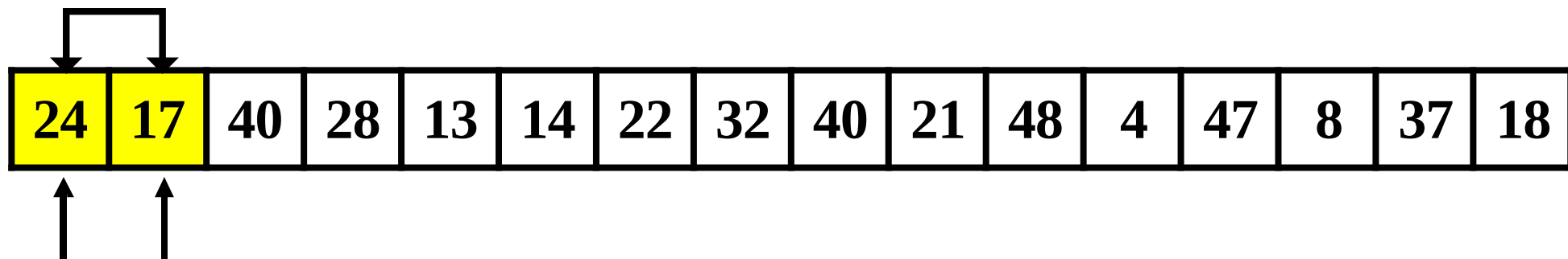
↑ ↑



选择排序



- 输入：<24, 17, 40, 28, 13, 14, 22, 32, 40, 21, 48, 4, 47, 8, 37, 18>



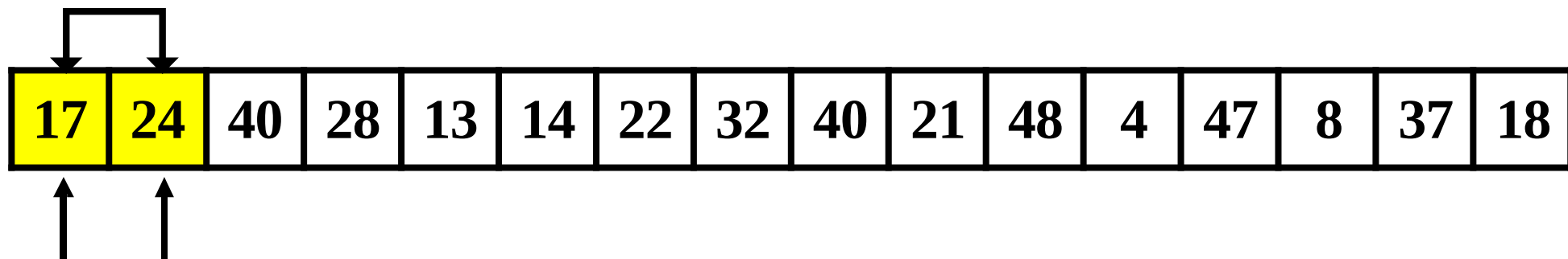
24 > 17



选择排序



- 输入：<24, 17, 40, 28, 13, 14, 22, 32, 40, 21, 48, 4, 47, 8, 37, 18>



$17 < 24$



选择排序



- 输入：<24, 17, 40, 28, 13, 14, 22, 32, 40, 21, 48, 4, 47, 8, 37, 18>

17	24	40	28	13	14	22	32	40	21	48	4	47	8	37	18
----	----	----	----	----	----	----	----	----	----	----	---	----	---	----	----

↑ ↑

17 < 40



选择排序



- 输入：<24, 17, 40, 28, 13, 14, 22, 32, 40, 21, 48, 4, 47, 8, 37, 18>

17	24	40	28	13	14	22	32	40	21	48	4	47	8	37	18
----	----	----	----	----	----	----	----	----	----	----	---	----	---	----	----

↑ ↑

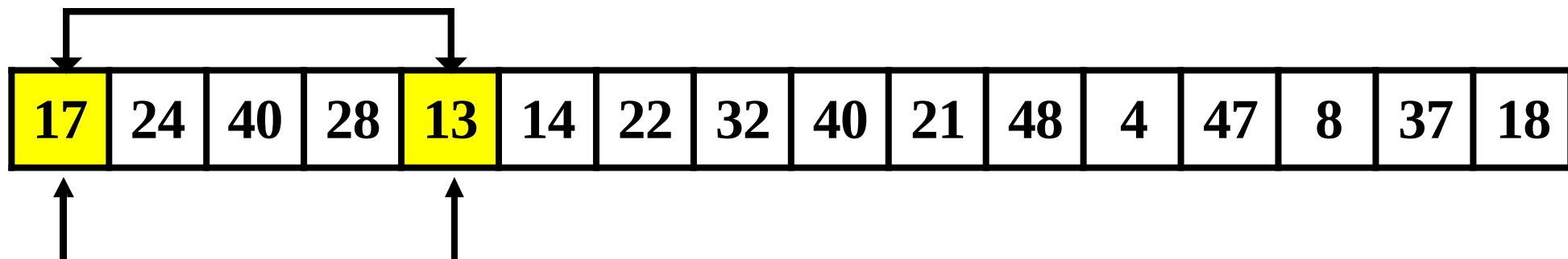
$17 < 28$



选择排序



- 输入：<24, 17, 40, 28, 13, 14, 22, 32, 40, 21, 48, 4, 47, 8, 37, 18>



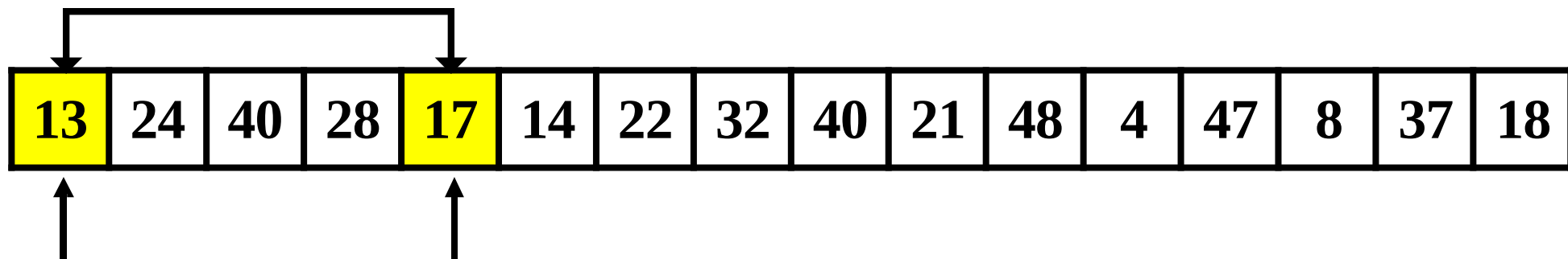
$17 > 13$



选择排序



- 输入：<24, 17, 40, 28, 13, 14, 22, 32, 40, 21, 48, 4, 47, 8, 37, 18>



$13 < 17$



选择排序



- 输入：<24, 17, 40, 28, 13, 14, 22, 32, 40, 21, 48, 4, 47, 8, 37, 18>

13	24	40	28	17	14	22	32	40	21	48	4	47	8	37	18
----	----	----	----	----	----	----	----	----	----	----	---	----	---	----	----

↑ ↑

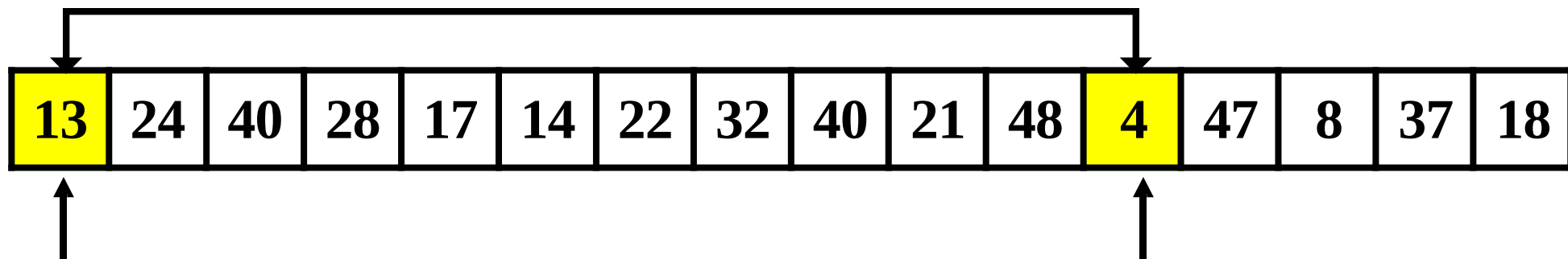
13 < 14



选择排序



- 输入：<24, 17, 40, 28, 13, 14, 22, 32, 40, 21, 48, 4, 47, 8, 37, 18>



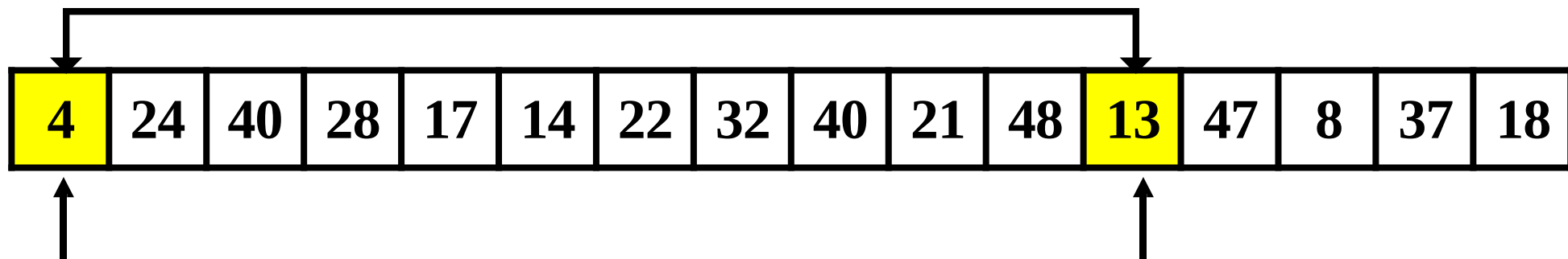
$13 > 4$



选择排序



- 输入：<24, 17, 40, 28, 13, 14, 22, 32, 40, 21, 48, 4, 47, 8, 37, 18>



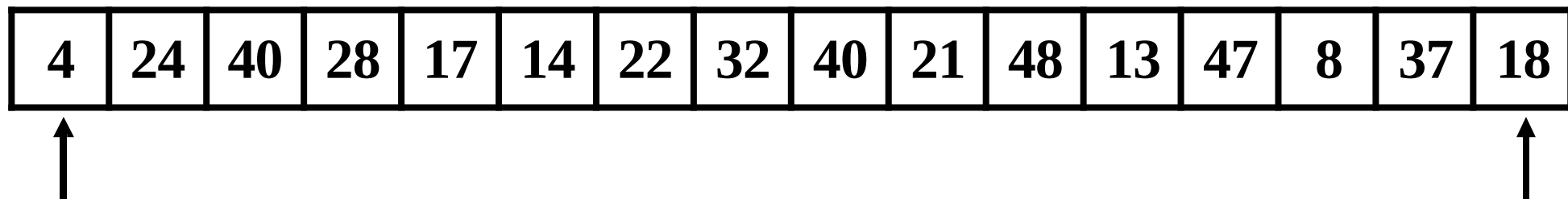
$4 < 13$



选择排序



- 输入：<24, 17, 40, 28, 13, 14, 22, 32, 40, 21, 48, 4, 47, 8, 37, 18>



$4 < 18$



选择排序



- 输入：<24, 17, 40, 28, 13, 14, 22, 32, 40, 21, 48, 4, 47, 8, 37, 18>

4	24	40	28	17	14	22	32	40	21	48	13	47	8	37	18
---	----	----	----	----	----	----	----	----	----	----	----	----	---	----	----



选择排序



- 输入：<24, 17, 40, 28, 13, 14, 22, 32, 40, 21, 48, 4, 47, 8, 37, 18>

4	8	40	28	24	17	22	32	40	21	48	14	47	13	37	18
---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----



选择排序



- 输入：<24, 17, 40, 28, 13, 14, 22, 32, 40, 21, 48, 4, 47, 8, 37, 18>

4	8	13	14	17	18	21	22	24	28	32	37	40	40	47	48
---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----



选择排序

- 输入： <24, 17, 40, 28, 13, 14, 22, 32, 40, 21, 48, 4, 47, 8, 37, 18>
- 输出： <4, 8, 13, 14, 17, 18, 21, 22, 24, 28, 32, 37, 40, 40, 47, 48>

4	8	13	14	17	18	21	22	24	28	32	37	40	40	47	48
---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----



插入和选择排序是算法吗？

- 输入：[7, 3, 4, 11, 10]
- 输出：[3, 4, 7, 10, 11]
- 选择排序算法描述：
 - 第一次遍历找到最小元素
 - 第二次在剩余数组中遍历找到次小元素
 - ...
 - 第n次在剩余数组中遍历找到第n小元素

- 良定义（确定性）
- 步骤有限（有穷性）
- 可机械操作（能行性）

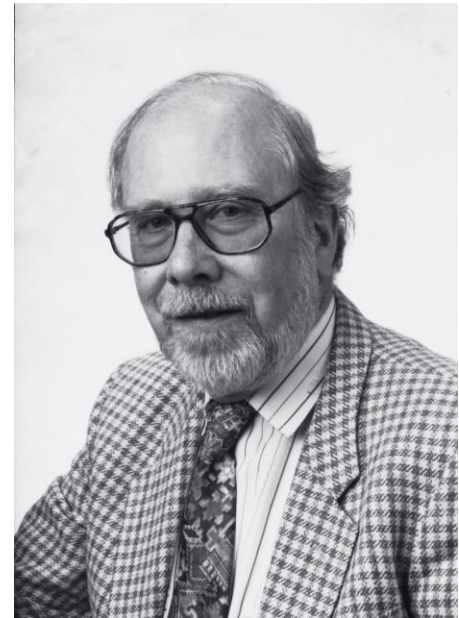
反例

- 对于给定输入数组，交换两个数的位置
- 给定输入数组，不断交换首尾元素的位置
- 将大元素放数组后部，小元素放数组前部



计算过程与算法的区别

- 计算过程可以不满足算法的有穷性
- 例如操作系统，当没有任务时，操作系统并不终止，而是处于等待状态，直到有新的任务启动，因而不是一个算法
- 程序 = 算法 + 数据结构 (By Niklaus Emil Wirth)



Niklaus Emil Wirth
Pascal之父



算法的两大重要问题

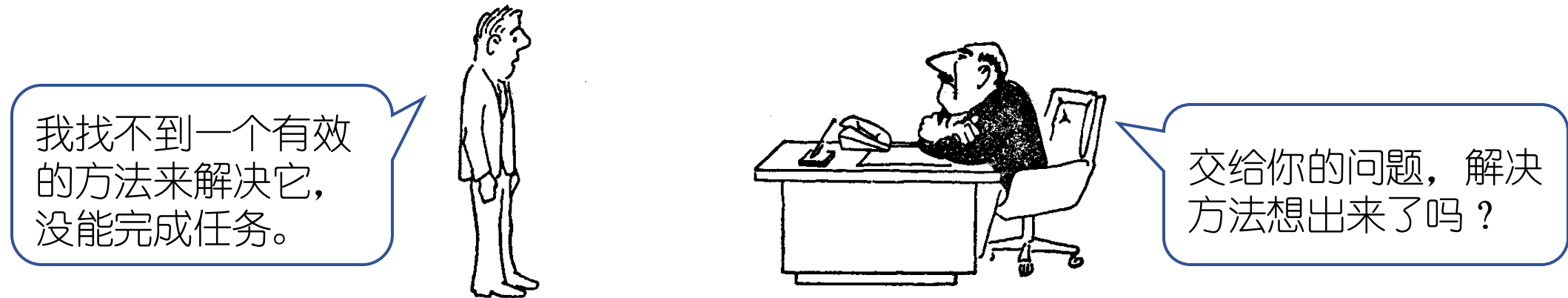
- 问题能解决吗？(可计算性)
- 问题解决的好吗？(计算复杂性)

用计算机（机械式）实现



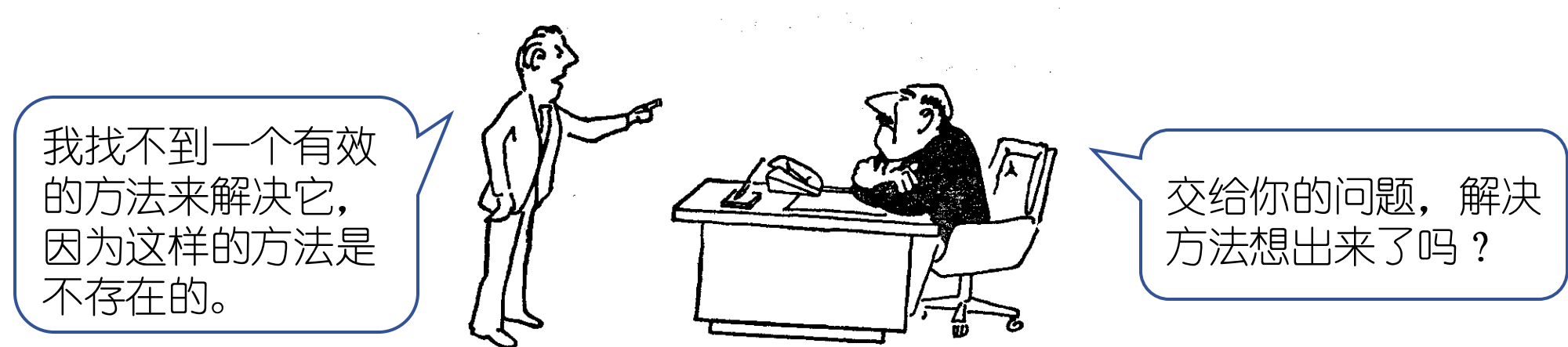
问题能解决么？

- 假设某一负责人交给你一个很难的任务，几天后询问你问题解决了没有。可能会发生如下图这样的情况：



问题能解决么？

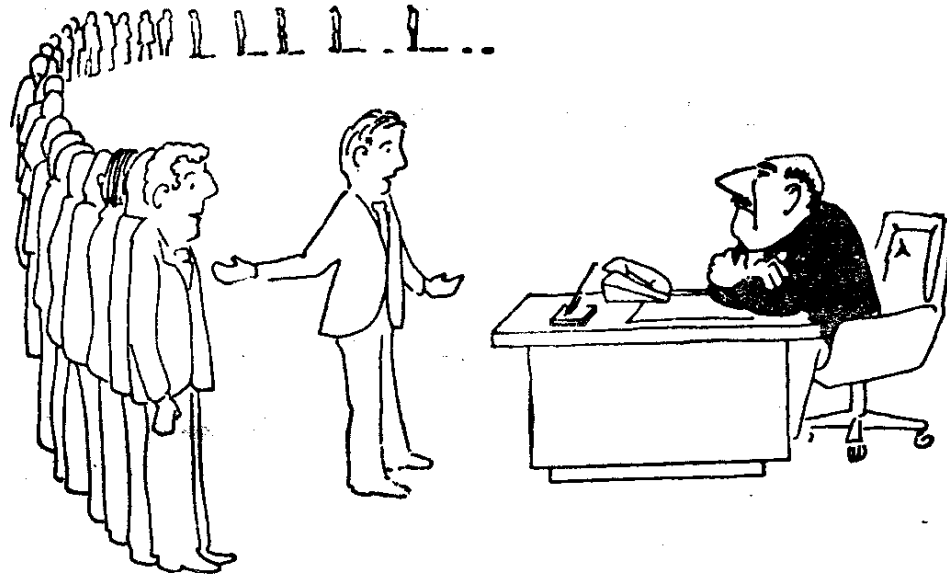
要证明一个问题不存在有效的方法，往往比寻找一种有效方法更难



问题解决的好吗？

方法消耗的资源太大了

我找到了一方法来
解决它，理论上可
实现的，但是以我
们目前的力量实现
它是不可能的。



交给你的问题，解决
方法想出来了吗？



可计算性研究的范畴

- 计算机虽然神通广大，还是在人的控制下工作
- 计算机并非什么都能做，有的事情理论上它根本做不了
- 讨论哪些事计算机能做，哪些事计算机做不了，属于可计算性理论研究的范畴。



一个满足可计算性的问题

- 26个英文字母全排列，它的排列数为

$$26! \approx 4 \times 10^{26}$$

- 以每年365天计算，共有

$$365 \times 24 \times 3600 = 3.1536 \times 10^7 \text{秒。}$$

- 以每秒能完成 10^7 个排列的超高速电子计算机来做这项工作，需要

$$4 \times 10^{26} / (3.1536 \times 10^7 \times 10^7) \approx 1.2 \times 10^{12} \text{年。}$$



算法分析研究的范畴

- 有一些问题虽然在理论上能够由计算机解决，但因其算法占用资源太多而无法实际完成，因此需要选择其他算法或改进算法。
- 要知道算法的优劣好坏，就需要对算法需要多少计算时间和存储空间做定量的分析。

是好算法则不必对其怀疑而再浪费时间进行研究。不是好算法则应再进行研究改进



本课程的计算机

- 本课程指的计算机是和图灵机计算能力等价的、冯诺伊曼体系结构计算机, 即**确定性图灵机**
 - 每步操作只能有一个结果!
 - 只需笔和纸即可实现
- 量子计算机是非确定性图灵机, 其算法和计算复杂性完全不同。



几个典型的非数值计算问题

- 货郎担问题(TSP问题) ←
- **[动态规划]** 设有 n 个城市，已知任意两城市间之距离，现有一售货员想从某一城市出发巡回经过每一城市（且每城市只经过一次），最后又回到出发点，问如何找一条最短路径。试一试求出最短路径。
- n 皇后问题
- 背包问题



几个典型的非数值计算问题

- 货郎担问题(TSP问题)
- n皇后问题 ←
- 背包问题
- [回溯法] 高斯 (1850年) : 国际象棋中的“皇后”在横向、竖向、和斜向都能走步和吃子, 在 $n \times n$ 格的棋盘上如何能摆上 n 个皇后而使她们都不能互相吃。
 - 当 n 很大时, 问题很难。
 - $n=8$ 时, 共有92种解, 但只有12种是独立的, 其余的由这12种利用对称性或旋转而得到。



几个典型的非数值计算问题

- 货郎担问题(TSP问题)
- n皇后问题
- 背包问题
- 有三种物品，物品1重10千克，价值60元；物品2重20千克，价值100元；物品3重30千克，价值120元。要从中选取不超过50公斤重的物品，且总价值最大。
 - [动态规划]物品不可分割的前提下，求总价值最大。
 - [贪心算法]物品可以分割的前提下，求总价值最大。





本章结束

